

经典 DP 及优化

清华大学 梁宸铭

所有计数题如果没有特殊说明，默认对 998244353 取模。

→ 和 ← 表示更新。最优化问题中，更新通常是取 $\max, \min$ ，计数问题中，更新通常是加法。

划分数

给定 n ，计数数列 $\{a_k\}$ 的数量，满足：

- $1 \leq a_k \leq n$ 。
- $\sum_k a_k = n$ 。
- $a_k \leq a_{k+1}$ 。

想一想，怎么做到 $\mathcal{O}(n^2)$ ？怎么做到 $\mathcal{O}(n\sqrt{n})$ ？

状态设计

相当于计数阶梯型的数量，使得阶梯型的面积为 n 。称这种阶梯型为本题的**组合对象**。

考虑两种生成阶梯型的方式。

- 方式 1：从左往右加柱子，对应 DP 状态： $dp_{i,j}$ 表示面积为 i ，最后一个柱子的高度为 j 。
- 方式 2：从下往上一层一层加，对应 DP 状态： $dp_{i,j}$ 表示面积为 i ，最下层的宽度为 j 。对应转移方程为 $dp_{i,j} = dp_{i-j,j} + dp_{i-1,j-1}$ 。

方式 1 暴力做是 $\mathcal{O}(n^3)$ 的（前缀和优化可以做到 $\mathcal{O}(n^2)$ ），而方式 2 暴力做就是 $\mathcal{O}(n^2)$ 的。

优化

运用一个性质：大于 $\sqrt{n}$ 的数不会超过 $\sqrt{n}$ 个。

对不超过 $\sqrt{n}$ 的数用方式 1，对超过 $\sqrt{n}$ 的数用方式 2。

[ABC219H] Candles

在一条无限延伸的数轴上，放置着 N 根蜡烛。第 i 根蜡烛位于坐标 X_i ，在时刻 0 时长度为 A_i ，并且已经点燃。被点燃的蜡烛每分钟长度减少 1，当长度变为 0 时就会燃尽，此后长度不再变化。此外，被熄灭的蜡烛长度也不会再发生变化。

高桥君在时刻 0 时位于坐标 0，每分钟最多可以移动 1 的距离。如果高桥君所在的坐标正好有蜡烛，他可以将该位置所有的蜡烛同时熄灭（熄灭蜡烛所需时间可以忽略不计）。

请你求出高桥君采取最优行动时，从时刻 0 到 10^{100} 分钟后，所有剩余蜡烛长度的总和可能达到的最大值。

$n \leq 300$ ，其它数据范围都在 10^9 内。

分析问题模型

问题的**组合对象**是一条经过所有蜡烛的路径，**目标函数**是蜡烛的长度和。

设到达蜡烛 i 的时间为 t_i ，则目标函数为

$$\sum_{i=1}^N \max(0, A_i - t_i)$$

需要最大化目标函数的值。

状态设计

如何表示**组合对象**？需要包含的信息有：当前在哪个蜡烛，已经经过了哪些蜡烛。

注意到任意时刻经过的蜡烛构成一个区间，所以可以采用区间 DP 的方式表示上述信息。设 $dp_{l,r,0/1}$ 表示已经经过了区间 $[l, r]$ 中的所有蜡烛，当前在 l 还是在 r 。

如何计算**目标函数**？一个朴素的方式是直接把当前时间 t 塞进状态，设 $dp_{l,r,t,0/1}$ 中的 t 表示当前经历了多少时间。

更好的状态设计

t 这一维太浪费了，怎么办？

考虑更优秀的计算方法。注意到我们每走长度为 x 的路，所有走完这段路之后没有熄灭的蜡烛长度都减少了 x 。

所以只要知道没有熄灭的蜡烛数量为 k ，就可以直接把答案减去 kx ！这个思想被称作**费用提前**。

但是有一个问题，怎么知道哪些蜡烛熄灭了？

我们可以通过**增加决策步骤**来解决这个问题。增加的决策步骤是：在一开始就选好哪些蜡烛没有灭。

为什么可以这么做？因为如果选错了，我们就需要接受一个“长度为负数”的蜡烛。如果选漏了，我们就白白少了一段蜡烛。

更好的状态设计

设 $dp_{l,r,k,0/1}$ 中的 k 表示在没有到达的蜡烛中，有 k 个是我们一开始就指定没有灭的。

$dp_{l,r,k,0}$ 的状态转移方程为

$$dp_{l,r,k,0} \leftarrow dp_{l,r,k,1} - k(x_r - x_l)$$

$$dp_{l,r,k,0} \leftarrow dp_{l+1,r,k,0} - k(x_{l+1} - x_l)$$

$$dp_{l,r,k,0} \leftarrow dp_{l+1,r,k+1,0} - (k+1)(x_{l+1} - x_l) + a_l$$

小结

思考一道 DP 问题一般有三步：

- 第一步：确定组合对象、限制、目标函数（最优化问题）。
- 第二步：确定生成组合对象、满足限制、计算目标函数的方法，把必要的值存进状态里。
- 第三步：根据题目性质优化 DP。

计算目标函数的小技巧：增加决策，费用提前。

Mex

给定 n 和数列 $\{b_i\}$ ，计数数列 $\{a_i\}$ 的个数，满足：

- $0 \leq a_i \leq n$ 。
- $\text{mex}(a_1, a_2, \dots, a_i) = b_i$ ，其中 $\text{mex}(a_1, a_2, \dots, a_i)$ 表示 $a_{1 \sim i}$ 中最小的没有出现过的自然数（例： $\text{mex}(2, 0, 1, 4) = 3$ ）

$n \leq 5000$ 。

强化版：CF1608F MEX counting

初步分析

组合对象是数列 a_i ，**限制**是前缀 mex 为指定值。

一种容易想到的生成组合对象的方式是从前往后一个一个填 a_i ，但这样难以满足前缀 mex 的限制。

更聪明的生成方法

注意到如果 i 前面的一个 a_j 比 mex (即 b_i) 还大, 它取什么值都不会影响 b_i 的值。

那么可以**延迟决策**, 如果当前的 a_i 对 b_i 没有影响, 那么先填上一个 `?`, 等到有影响的时候再决定具体值。

因为相同值只需要考虑它第一次出现的位置, 所以如果一个位置不是第一次出现, 那么我们可以指定它和前面哪个问号相同, 等到填上问号值的时候把它顺便填上即可。

状态转移

设 $dp_{i,j}$ 表示填到第 i 个数，前面填了 j 个值两两不同的问号。状态转移为

如果 $b_i = b_{i+1}$

$$b_i \cdot dp_{i,j} \rightarrow dp_{i+1,j}$$

$$dp_{i,j} \rightarrow dp_{i+1,j+1}$$

$$j \cdot dp_{i,j} \rightarrow dp_{i+1,j}$$

如果 $b_i \neq b_{i+1}$ ，此时 a_{i+1} 必然和 b_i 相等，再指定 $b_{i+1} - b_i - 1$ 个问号补上 $b_i \sim b_{i+1} - 1$ 的部分，用排列组合的方式计算就是

$$A_j^{b_{i+1}-b_i-1} dp_{i,j} \rightarrow dp_{i+1,j-(b_{i+1}-b_i-1)}$$

P6326 Shopping

马上就是小苗的生日了，为了给小苗准备礼物，小葱兴冲冲地来到了商店街。商店街有 n 个商店，并且它们之间的道路构成了一棵树的形状。

第 i 个商店只卖第 i 种物品，小苗对于这种物品的喜爱度是 w_i ，物品的价格为 c_i ，物品的库存是 d_i 。但是商店街有一项奇怪的规定：如果在商店 u, v 买了东西，并且有一个商店 p 在 u 到 v 的路径上，那么必须要在商店 p 买东西。小葱身上有 m 元钱，他想要尽量让小苗开心，所以他希望最大化小苗对买到物品的喜爱度之和。

$n \leq 500, m \leq 4000, d_i \leq 2500$ 。

初步分析

这是一个树上的多重背包问题，**组合对象**是连通块和购买方案。

树上背包的一个经典生成方式是自下而上，每次合并子树的连通块。具体地，设 $dp_{u,i}$ 表示以 u 为根的子树的连通块，价格总和为 i 的最大喜爱度。对于 u 的一个儿子 v ，转移为

$$dp_{u,i} + dp_{v,j} \rightarrow dp_{u,i+j}$$

复杂度是 $\mathcal{O}(nm^2)$ 的，相当高。

更聪明的生成方式

考虑一个弱化版的问题，如果最后的连通块包含 1，应该怎么生成？

可以从点 1 自上而下，按照 dfs 序 DP。

具体地，考虑 dfs 序中的第 i 个点 u ，如果加入它，那么就继续考虑 dfs 序中的第 $i + 1$ 个点；如果不加入它，那么它子树内的所有点都不会被加入，此时跳过它，考虑子树外的第一个点，即 dfs 序中第 $i + \text{size}_u$ 个点。

这样生成的好处是，每次只需要加入一个点，而不是一整个连通块。

状态转移

多重背包可以通过二进制拆分转化为 01 背包。设 $dp_{i,j}$ 表示考虑到 dfs 序中的第 i 个点，价格为 j ，转移为

$$\begin{aligned} dp_{i,j} &\rightarrow dp_{i+\text{size}_u,j} \\ dp_{i,j} + w &\rightarrow dp_{i+1,j+c} \end{aligned}$$

这样，我们就解决了连通块中包含 1 的问题，复杂度优化到了 $\mathcal{O}(nm \log d_i)$ 。

如何扩展到不包含 1 的情况？可以点分治，每次对分治中心做上面的 DP，这样每个连通块都会在分治中心为连通块包含的点分树上最浅的点时被统计到。由于不是本节课的重点，此处略过。最终复杂度为 $\mathcal{O}(nm \log n \log d_i)$

P6773 [NOI2020] 命运（弱化版）

给定一个 n 个点的树和 m 条限制，你可以给树上的一条边赋一个 0 或 1 的权值。对于所有限制 (u, v) （**保证 v 为 u 的祖先**），你需要保证 u 到 v 上至少有一条边的权值为 1，求赋值方案数。

$n \leq 5000$ 。

初步分析

组合对象为给边赋权的方式，**限制**为路径上至少有一条权值为 1 的边。

生成对象的方式是简单的，自下而上地确定每条边的权值即可。关键是如何满足限制。

满足限制的方法

假设我们自下而上考虑到了 u ，设经过 (u, fa_u) ，且在 u 的子树中仍然没有被“解决”的限制集合为 S 。

一种粗暴的方法是把 S 集合全部塞进状态里面。但我们可以找到所有限制中“最关键的”那一个。可以发现，端点最深的那个限制是最关键的，因为只要满足了它，其它限制必然会被满足。所以我们不妨记录最深端点的深度。

状态设计和转移

设 $f_{u,i}$ 表示考虑到 u ，但还没决定 (u, fa_u) 的取值，子树中还未被解决的限制中端点最深的深度为 i 的方案数。 $g_{u,i}$ 表示考虑到 u ，并且决定了 (u, fa_u) 的取值的方案数。

则转移为

$$\begin{aligned} f_{u,i} &\rightarrow g_{u,i}(\text{dep}_u \geq i + 1) \\ f_{u,i} &\rightarrow g_{u,-\infty} \end{aligned}$$

设 v 为 u 的子结点

$$f_{u,i} g_{v,j} \rightarrow f_{u, \max(i,j)}$$

复杂度为 $\mathcal{O}(n^2)$ ，使用线段树合并优化可以做到 $\mathcal{O}(n \log n)$ 。这种优化技巧也被称作整体 DP，感兴趣的同学可以自己学习。

P9197 [JOI Open 2016] 摩天大楼 / Skyscraper

将互不相同的 n 个整数 $a_1, a_2, \dots, a_n$ 按照一定顺序排列。

假设排列为 $f_1, f_2, \dots, f_n$ ，要求 $\sum_{i=1}^{n-1} |f_i - f_{i+1}| \leq L$ 。

求满足题意的排列的方案数对 $10^9 + 7$ 取模后的结果。

初步分析

组合对象为排列，**限制**为相邻数的绝对值之和不超过 L 。

排列的常见生成方式有两种，一种是从前往后，一个一个按照大小关系插入之前的排列。一种是按照值从小到大加入数。

选择哪种生成方式关键在于哪种方式更方便满足题目的限制。

从前往后加是比较难以计算相邻数绝对值的和的，因为在插入一个数的同时，我们需要知道有多少对相邻的数满足一个大于它，另一个小于它。

生成方法

我们选择值从小往大的生成方式。

定义 sum_of_abs 初始为 0。在加入一个数 i 后假设有 t 对相邻的数满足其中一个数已经被加入，另一个数没有被加入，那么我们令 $\text{sum_of_abs} \leftarrow \text{sum_of_abs} + t$ ，则可以证明加入所有 n 个数之后：

$$\text{sum_of_abs} = \sum_{i=1}^{n-1} |f_{i+1} - f_i|$$

证明：假设 $f_i < f_{i+1}$ ，那么在加入 $f_i, f_i + 1, \dots, f_{i+1} - 1$ 的时候， i 和 $i + 1$ 这对数都处在一个已经被加入，一个还没有被加入的状态，故这对数在 sum_of_abs 恰好被算了 $|f_{i+1} - f_i|$ 次。

状态设计和转移

为了计算 `sum_of_abs`，我们采用一种叫做**连续段 DP**的方法。设 $dp_{i,j,k,x,y}$ 表示从小到大填到 i ，有 j 个连续段，`sum_of_abs` 为 k ，开头是否被加入，结尾是否被加入，那么有等式

$$t = 2j - x - y$$

转移有三种：新开一个连续段，合并两个连续段，延续其中一段。转移方程较为复杂，大家课后可以自行思考。最终时间复杂度为 $\mathcal{O}(n^2 L)$ 。

小结

- 分析了常见组合对象的处理方式：序列（排列）、树（连通块）
- 两个经典套路：dfs 序 DP，连续段 DP。

[ABC213G] Connectivity 2

给一张 n 个点 m 条边的简单无向图 G ，考虑删去 0 条及以上的边构成一张新图，对每个点 k ，求有多少张新图满足点 k 与点 1 连通。

初步分析

组合对象是子图。

如果正着考虑，生成一张连通图相当困难。但是生成一张不连通的图却比较简单，我们可以把原图分成两个部分：与点 u 联通的点，与点 u 不联通的点，然后钦定两个点集之间没有任何边。

这告诉我们，可以采用**容斥**计算图的数量。

状态设计

设 $f(S)$ 表示点集为 S 的联通子图的数量， $g(S)$ 表示点集为 S 的子图的数量。设点集 S 的导出子图的边数为 cnt ，则 $g(S) = 2^{cnt}$ 。任取 $u \in S$ ，则转移为

$$f(S) = g(S) - \sum_{u \in T, T \subsetneq S} f(T)g(S \setminus T)$$

复杂度为 $\mathcal{O}(3^n)$ 。

P5405 [CTS2019] 氪金手游

简化版题意：给定一棵 n 个点的有向树，计数排列 $\{p_i\}$ 的数量，满足

- 对于任意一条有向边 (u, v) ， $p_u < p_v$ 。

提示：如果给定的有向树为以 1 为根的外向树，则排列数量为

$$n! \prod_{i=1}^n \frac{1}{\text{size}_i}$$

链上的版本：「LibreOJ NOI Round #2」不等关系

分析

假设树以 1 为根，那么边分为向上的边和向下的边。如果所有边都是向下的边，则可直接根据公式计算答案。扩展到一般情况，我们能否把问题转化为所有边都向下的情况？

对向上的边容斥，假设向上的边集合为 S ，钦定其中的一个子集 T 中的限制都不满足，假设新问题的答案为 $f(T)$ ，则原问题的答案为

$$\sum_{T \subseteq S} (-1)^{|T|} f(T)$$

再分析这个问题：

- 对于集合 T 中的边，因为钦定这些限制不满足，所以边的方向被反转，所有边都变成了向下的边。
- 对于集合 $S \setminus T$ 中的边，没有限制，相当于删掉了这些边。

此时树上的一些边被删掉了，且所有的边都是向下的边。对于新得到的森林，其方案数为

$$n! \prod_{i=1}^n \frac{1}{\text{size}_i}$$

用 DP 计算容斥系数

此时可以用 DP 的视角看待这个问题。

组合对象为容斥选择的集合，一个组合对象的**权重**为

$$n!(-1)^{|T|} \prod_{i=1}^n \frac{1}{\text{size}_i}$$

采用 DP 计算这个权重。

设 $dp_{u,i}$ 表示对于 u 的子树, $\text{size}_u = i$ 时, 权重的和。设 v 是 u 的儿子, 转移为

$$\begin{aligned} dp_{u,i} \cdot dp_{v,j} &\rightarrow dp_{u,i+j} \\ -dp_{u,i} \cdot dp_{v,j} &\rightarrow dp_{u,i} \end{aligned}$$

最后计算 u 的子树大小对权重的贡献, 即

$$dp_{u,i} \leftarrow dp_{u,i} \cdot \frac{1}{i}$$

最后乘上 $n!$ 就是总的权重。

CF1810G The Maximum Prefix

你需要生成一个长度不超过 n ($1 \leq n \leq 5000$) 的数组 a ，其中每个 a_i 的取值为 1 或 -1 。

你按照如下方式生成该数组：

- 首先，你选择一个整数 k ($1 \leq k \leq n$)，决定数组 a 的长度。
- 然后，对于每个 i ($1 \leq i \leq k$)，你以概率 p_i 令 $a_i = 1$ ，否则以概率 $1 - p_i$ 令 $a_i = -1$ 。

数组生成后，你计算 $s_i = a_1 + a_2 + a_3 + \dots + a_i$ ，特别地， $s_0 = 0$ 。然后令

$S = \max_{i=0}^k s_i$ ，即 S 是数组 a 的最大前缀和。

你会得到 $n + 1$ 个整数 $h_0, h_1, \dots, h_n$ 。对于最大前缀和为 S 的数组 a ，其得分为 h_S 。现在，对于每个 k ，你需要求出长度为 k 的数组的期望得分。

初步分析

组合对象为序列，权重为 $h_S \cdot p(a)$ ， S 为序列的最大前缀和， $p(a)$ 为序列 $\{a_i\}$ 的概率。

本题最大的问题在于如何计算权重。采用**差分**技巧。设 $f(S)$ 表示最大前缀和为 S 的序列数量，则根据 Abel 变换，答案为

$$\sum_{S=0}^n f(S) h_S = h_n \sum_{S=0}^n f(S) - \sum_{k=0}^{n-1} (h_{k+1} - h_k) \sum_{S=0}^k f(S)$$

令

$$g(k) = \sum_{S=0}^k f(S), \Delta h_k = \begin{cases} h_k - h_{k+1} & k < n \\ h_n & k = n \end{cases}$$

$g(k)$ 的组合意义为最大前缀和小于等于 k 的序列数量，则答案为

$$\sum_{k=0}^n \Delta h_k g(k)$$

所以通过差分，我们把最大前缀和等于 S 的限制转化为了最大前缀和小于等于 S 的限制。

DP 与优化

枚举 k ，并设 $dp_{i,j}$ 表示考虑前 i 个，且位置 i 的前缀和为 j ，并且全过程中，最大前缀和没有大于 k 的概率。假设最后得到的总概率为 p ，把 $p \cdot \Delta h_k$ 累加进答案。由于需要做 n 次 DP，复杂度为 $\mathcal{O}(n^3)$ 。

注意到 n 次 DP 的过程特别类似，我们能否通过操作使得只做一次 DP？

操作如下：

- **统一上下界**：假设初始前缀和为 k ，把加看成减，减看成加，则原来上界为 k 的限制变为下界为 0。
- **将权重放入初值**：原来初值为 $dp_{0,k} = 1$ ，改成 $dp_{0,k} = \Delta h_k$ ，这样最后答案不需要额外乘上 Δh_k 。

此时 n 次 DP 过程完全一样，直接放在一起做。具体地，对于 $k = 0 \sim n$ ，初始化 $dp_{0,k} = \Delta h_k$ ，然后化加为减，化减为加，并限制下界为 0。最后的答案就是 $dp_{n,i}$ 的和。

复杂度为 $\mathcal{O}(n^2)$ 。

此题题解区中的“反推贡献系数”（或称“反转 DP”）也很有启发性，推荐大家阅读。这个方法和预习版课件中的 Merging Cells 和 Tree Topological Order Counting 两题很有关系。

小结

- DP 可以和容斥相结合。
- 上面的内容基本覆盖了 DP 状态设计的几个关键问题：组合对象，限制，目标函数（最优化问题），权重（计数问题）。大家在总结 DP 方法的时候，可以从上面几个角度出发。

通过折线性质优化 DP

- 斜率优化。
- 四边形不等式优化。
- 凸性优化。

斜率优化和四边形不等式优化

可以使用斜率优化和四边形不等式优化的 DP 基本为如下形式

$$dp_j = \min_{i < j} (dp_i + w(i, j))$$

其中斜率优化的 $w(i, j)$ 在图像上表示为一条直线，即

$$w(i, j) = k_i x_j + b_i$$

四边形不等式优化中 $w(i, j)$ 满足

$$w(i + 1, j + 1) - w(i + 1, j) \leq w(i, j + 1) - w(i, j)$$

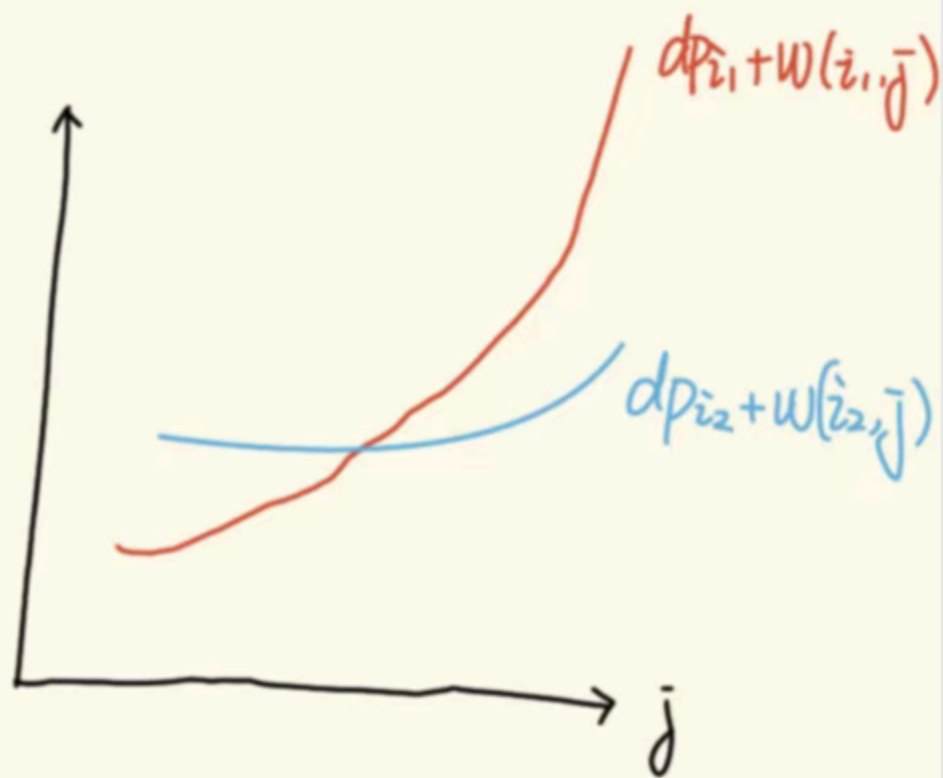
用文字描述就是**越靠后的增长的越慢**。也就是说，如果对于 $i_1 < i_2$ ，

$dp_{i_1} + w(i_1, j) > dp_{i_2, j} + w(i_2, j)$ ，那么对于所有 $j' > j$ ， i_2 都是比 i_1 更优的决策点。这就是决策单调性的由来。

斜率优化



四边形不等式优化. ($i_1 < i_2$)



决策单调性分治

问题：给定满足四边形不等式的二元函数 $w(i, j)$ 和数列 f ，在 $\mathcal{O}(n \log n)$ 的时间内求出数列 g

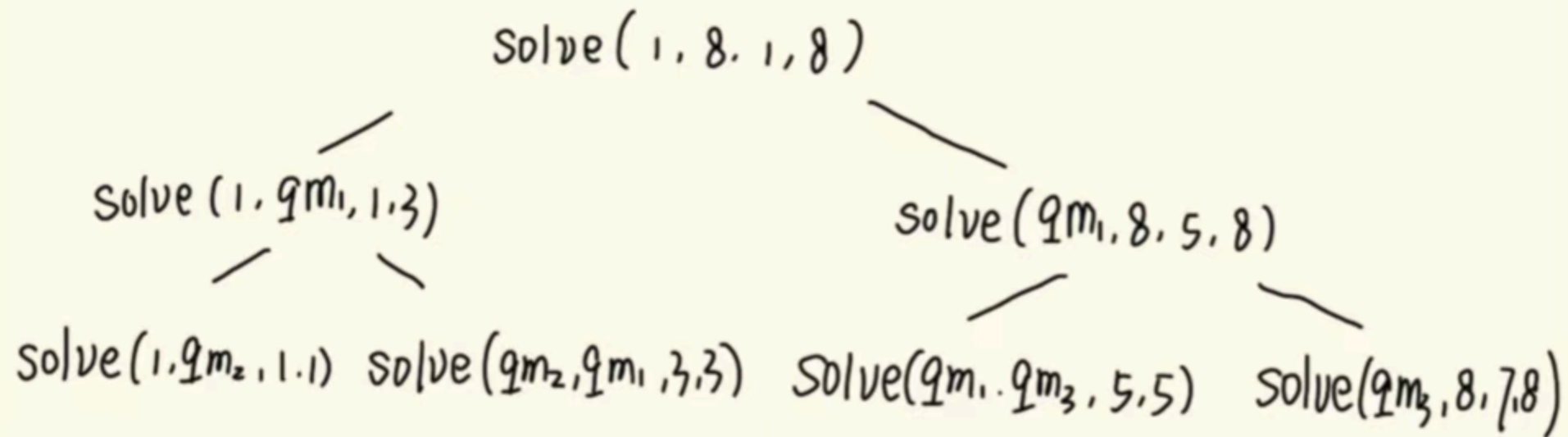
$$g_j = \min_{i < j} (f_i + w(i, j))$$

假设 $\text{solve}(ql, qr, l, r)$ 表示需要求出 $g_{l \sim r}$ ，且确定最优决策点在 $[ql, qr]$ 中。初始时调用 $\text{solve}(1, n, 1, n)$ 。

对于每个 $\text{solve}(ql, qr, l, r)$ ，令 $m = (l + r)/2$ 。

- 遍历 $[ql, qr]$ ，求出 g_m 的最优决策点 qm 。
- 递归调用 $\text{solve}(ql, qm, l, m - 1)$ ， $\text{solve}(qm, qr, m + 1, r)$ 。

每次递归的复杂度为 $\mathcal{O}(qr - ql + 1)$ 。对于递归的每一层， $[ql, qr]$ 是不交的，和至多为 n ，且递归只有 $\mathcal{O}(\log n)$ 层，所以总复杂度为 $\mathcal{O}(n \log n)$ 。



P5574 [CmdOI2019] 任务分配问题

给定长度为 n 的排列 $\{a_n\}$ ，将排列分为 k 段，让每段内顺序对总和最小。

$n \leq 2.5 \times 10^4, k \leq 25$ 。

证明四边形不等式

设 $dp_{i,t}$ 表示前 i 个，分 t 段，段内顺序对总和的最小值。设 $w(i, j)$ 为 $[i, j]$ 的顺序对数量，则

$$dp_{i,t} = \min_{j < i} (dp_{j,t-1} + w(j+1, i))$$

由于顺序对数目满足**越靠后的增长越慢**（因为靠后的连续段中包含的数更少，加入一个新数之后，产生的顺序对更少），所以 $w(i, j)$ 满足四边形不等式。

$w(i, j)$ 的计算

把决策单调性分治做 k 遍就可以完成 DP 过程。现在唯一的问题在于 $w(i, j)$ 如何快速计算。

采用类似莫队的做法，可以分析单次递归指针的移动次数为 $\mathcal{O}((qr - ql + 1) + (r - l + 1))$ 。所以指针移动的总次数为 $\mathcal{O}(n \log n)$ ，如果采用树状数组，单次移动指针的复杂度为 $\mathcal{O}(\log n)$ 。故总复杂度为 $\mathcal{O}(kn \log^2 n)$ 。

决策单调性分治是一个离线的算法，如果需要在线地优化转移

$$f_j = \min_{i < j} (f_i + w(i, j))$$

可以使用二分栈算法，感兴趣的同学可以自行学习。

凸性优化 DP

有时，DP 数组 dp_i 会具有凸性（在可以用费用流模型来描述的问题中尤其常见）：

$$dp_i - dp_{i-1} \leq dp_{i+1} - dp_i$$

凸性的好处在于可以在很快的时间内完成 $(\min, +)$ 卷积（例：树上背包的合并过程），即对于两个长度为 n 的有凸性的数组 f_i, g_i ，可以在 $\mathcal{O}(n)$ 的时间内求出

$$h_i = \min_{k \leq i} (f_i + g_{k-i})$$

如果使用左偏树，甚至可以做到 $\mathcal{O}(\log n)$ 单次合并。

闵可夫斯基和

闵可夫斯基和是一个计算几何中的概念，有时它也可以用来表示两个凸数组的 $(\min, +)$ 卷积的结果，有结论

- 两个凸数组的 $(\min, +)$ 卷积的差分为两个数组的差分归并排序之后的结果。

感性理解：在店铺 A 一共买 i 件物品要花费 f_i ，第 i 件物品要花费 $f_i - f_{i-1}$ ；在店铺 B 一共买 i 件物品要花费 g_i ，第 i 件物品要花费 $g_i - g_{i-1}$ 。在两个店铺中一共买 k 件物品的最小花费即为 h_k ，它是贪心地在两个店铺中按照价格从小到大买物品的结果。

如果暴力归并排序，复杂度为 $\mathcal{O}(n)$ 。如果用左偏树维护差分数组，复杂度为 $\mathcal{O}(\log n)$ 。

P9168 [省选联考 2023] 人员调度（弱化版）

给定 n 个点的有根树，有 k 个人，第 i 个人在 u_i 上，有权值 v_i ，定义一棵树的价值为所有节点上的人的权值的最大值之和，现在可以将每个人放在其子树内的任意一个点上，求所有方案中树的价值最大值。

$n \leq 10^5$ 。

问题可以看作每个人占用一个点，求占用点的人的权值最大值的和。设 $dp_{u,i}$ 表示 u 子树内有 i 个点被占用的最大价值，设 v 为 u 的儿子，转移为

$$dp_{u,i} + dp_{v,j} \rightarrow dp_{u,i+j}$$

假设点 u 上的所有人的权值从大到小排序为 $v_1, v_2, \dots, v_m$ ，其前缀和数组为 $\{s_i\}$ (s_i 为凸)，对于每个人，有转移

$$dp_{u,i} + s_j \rightarrow dp_{u,i+j}$$

注意到所有转移都是 $(\max, +)$ 卷积的形式，根据归纳法可知， $dp_{u,i}$ 是下凸的。

可以用左偏树维护差分数组，对于 u ，需要做以下操作：

- 设 v 为 u 的儿子，合并所有 v 的左偏树。
- 将 $v_1, v_2, \dots, v_m$ 依次插入左偏树。
- 把左偏树删到只剩下不超过 size_u 个元素。

复杂度为 $\mathcal{O}(n \log n)$ 。

（容易发现整个过程和本题题解区的贪心算法是完全相同的）

（这种 DP 方式可以用来优化费用流问题，详见陈扩文：《浅谈 OI 中的模拟费用流问题》，2022 年国家集训队论文集）

补充习题

- [CSP-S 2019] Emiya 家今天的饭
- P4229 [清华集训 2017] 某位歌姬的故事
- P7519 [省选联考 2021 A/B 卷] 滚榜
- CF1327F AND Segments
- ABC261H Game on Graph
- [AGC013D] Piling Up
- [ZJOI2016] 小星星
- AT4352 [ARC101C] Ribbons on Tree
- loj3688. 「JOISC 2022 Day2」复制粘贴 3

补充习题（续）

- AGC009E Eternal Average
- ARC163D Sum of SCC
- CF1842G Tenzing and Random Operations
- CF1799H Tree Cutting
- CF1750F Majority
- [WC2021] 表达式求值
- 「USACO 2019.12 Platinum」 Tree Depth
- AT_arc112_e [ARC112E] Cigar Box
- CF375E Red and Black Tree

Reference

- dp 题方法总汇, <https://www.luogu.com/article/ki71nw88>
- OI TRICKS, <https://www.cnblogs.com/wyb-sen/p/17234499.html>
- zxy的思维技巧, <https://www.cnblogs.com/C202044zxy/p/15126199.html>
- 浅谈一类优化思想: 费用提前计算, <https://www.luogu.com.cn/article/bvovtmzp>
- [做题笔记] 浅谈状压dp在图计数问题上的应用,
<https://www.cnblogs.com/C202044zxy/p/15120848.html>
- 浅谈一类处理状态转移依赖邻项的排列计数问题的 dp 策略: 连续段 dp (线头 dp)
<https://www.cnblogs.com/chroneZ/p/17938137>
- 关于凸性的 things (wqs + slope trick + 闵可夫斯基和),
<https://www.luogu.com.cn/article/9zlb1psz>